

NightShift — Technical Design Document (TDD)

- NightShift — Technical Design Document (TDD)
 - RUBRIC CONTEXT (full traceability map lives in the PRD, §0)
 - SECTION 2: TECHNICAL DESIGN DOCUMENT (TDD)
 - 2.1 Tech Stack & Infrastructure
 - 2.2 Agent Tools & Interoperability (Day 2 Standards)
 - 2.2.1 Data Flow Example — the hard case through the full pipeline
 - 2.3 Context Engineering: Sessions, Skills & Memory (Day 3 Standards)
 - 2.4 Agent Quality, Security & Guardrails (Day 4 Standards)
 - 2.5 Prototype to Production: Spec-Driven Development (Day 5 Standards)
 - 2.6 Deployment Evidence (optional — counts toward Technical Implementation, not a separate category)
 - 2.7 Architecture Decision Log
 - 2.8 Build Priority Order (for the 8-day window)

NightShift — Technical Design Document (TDD)

Google & Kaggle 5-Day AI Agents Intensive (Vibe Coding) — Capstone Track: Agents for Business

Companion document: **NightShift — Product Requirements Document (PRD)**, covering value proposition, scope, success metrics, competitive positioning, and submission deliverables. This TDD covers the “how”; the PRD covers the “what and why.” Cross-references to “the PRD” throughout this document point to that companion file.

Revision: v10 (split) — this is the TDD half of a combined v10 PRD/TDD suite, split into two standalone files at the user’s request. Content is unchanged from v10; only the file boundary and a few cross-references changed.

RUBRIC CONTEXT (full traceability map lives in the PRD, §0)

For quick reference while implementing: **Technical Implementation (50 rubric points)** is scored primarily against the content of this document — §2.1 through §2.5 in full, with graph topology specifically in §2.2. **Deployment evidence (§2.6) is optional** and folds into this same 50-point category rather than being scored separately. The full Key Concepts checklist, Writeup/Video/README requirements, and legal constraints live in the PRD — see that document’s §0 and §3 if you need them while building.

⚠ **Do not include any API keys or passwords in your submitted code** — named explicitly as a disqualifying risk in Kaggle’s official rubric, not just general hygiene.

SECTION 2: TECHNICAL DESIGN DOCUMENT (TDD)

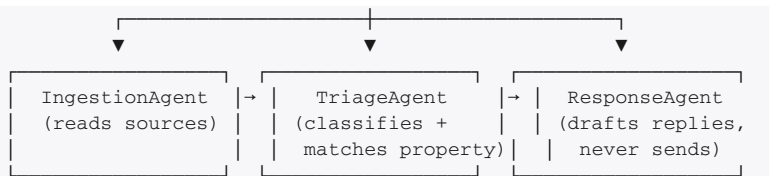
2.1 Tech Stack & Infrastructure

- **Core orchestration:** Python with Google Agent Development Kit (ADK 2.0), stateful code-first graph architecture. v1 ships as a **multi-agent supervisor graph** — three distinct ADK sub-agents (`IngestionAgent` , `TriageAgent` , `ResponseAgent`) coordinated by a lightweight supervisor node — rather than a single agent with internal skill modules. This is a deliberate design choice: the course’s mandatory features checklist names multi-agent orchestration explicitly as a graded item, and the natural task decomposition here (ingest → classify → draft) maps cleanly onto separate agents rather than one agent calling three internal functions. See §2.7 for the rubric-driven rationale.
- **Foundational LLM:** Gemini 1.5 Pro/Flash via Google AI Studio API. Flash for high-volume classification (cheap, fast), Pro reserved for drafting and ambiguous-case reasoning.
- **Secrets management:** Gemini API key and all third-party credentials are never hardcoded. Local dev uses a `.env` file (git-ignored); staging/production uses Google Cloud Secret Manager, injected at runtime as environment variables into the Cloud Run service. No secret is ever logged, including in OpenTelemetry trace attributes (explicit redaction list — see §2.4).
- **Development & deploy vector:** Built in Cursor IDE, run locally via `agents-cli` , containerized for staging deployment on Google Cloud Run.

2.2 Agent Tools & Interoperability (Day 2 Standards)

Multi-agent graph topology (Mandatory Feature #1 — Multi-Agent Orchestration):

SupervisorNode
(routing logic)



Each box is a distinct ADK agent with its own prompt, tools, and responsibility boundary — not a function call inside one agent. The `SupervisorNode` is plain routing code (no LLM call) that passes each overnight item through the pipeline and tracks per-item state. This separation is what the rubric’s “graph orchestration logic” line item is evaluating: a judge should be able to look at the code and see three independently testable agents, not one monolithic prompt doing everything.

- `IngestionAgent` — owns all MCP tool calls (below), produces structured `RawItem` objects.
- `TriageAgent` — consumes `RawItem`, calls Gemini Flash for classification, outputs `ClassifiedItem` with urgency tier + matched property.
- `ResponseAgent` — consumes `ClassifiedItem`, calls Gemini Pro to draft a response where applicable, writes to the staging store (never sends).

Per-item failure isolation (a real gap, not previously addressed): the design above processes items individually, which is correct for context-window cleanliness (§2.3) but raises an obvious operational question — what happens when item #4 of 50 fails (a Gemini timeout, a malformed mock fixture, a transient MCP error)? The `SupervisorNode` must not let one failing item crash the whole overnight run:

- Each item is processed inside its own try/except at the supervisor level; a failed item is logged with its error and marked `trriage_failed` in the staging store, not silently dropped and not allowed to halt the batch.
- A failed item still appears in the Morning Brief, flagged distinctly (e.g., “could not classify — needs manual review”) rather than disappearing — a missing item is worse than a clearly-flagged failed one, since the whole point of NightShift is that nothing falls through the cracks overnight.
- A small bounded retry (e.g., one retry on a transient API error before marking `trriage_failed`) is reasonable; retrying indefinitely is not, since a consistently-failing item should surface to the manager rather than loop silently.

Throughput vs. context-cleanliness (a real tension worth naming, not resolving away): processing items individually (§2.3) keeps each LLM call’s context small and accurate, but a 50-100 item overnight batch run strictly serially could risk exceeding the <10-minute Time-to-Morning-Brief target (PRD §1.3). The resolution is concurrency at the *item* level, not batching at the *prompt* level — these are different things and shouldn’t be conflated: `TriageAgent` still receives one item per call (preserving the context-cleanliness argument), but the `SupervisorNode` dispatches a small number of items concurrently (e.g., 3-5 in flight at once) rather than strictly one-at-a-time. This is a scheduling change in the supervisor, not an architecture change in any agent, and is worth treating as a v1 feature rather than an optimization to defer — it’s cheap to build now and directly protects a named success metric.

Model Context Protocol (MCP) (Mandatory Feature #2 — Tool Usage & Interoperability):

- A dedicated MCP server configuration exposes three tools, called exclusively by `IngestionAgent`: `read_inbox`, `read_hoa_portal`, `read_invoices_folder` — backed in v1 by mock endpoints (a local file stream for email, a stub REST API for the HOA portal, a watched folder for invoice PDFs).
- **This version reads from a mock inbox. It’s built so a real Gmail or Outlook connection could be swapped in later without changing the rest of the agent** — the `read_inbox` tool’s contract (input: none/since-timestamp; output: a list of structured `RawItem` objects) is identical whether the implementation behind it is a local file or a real Gmail API / Outlook Graph API call, so swapping the backend means changing one tool implementation, not the agent, the graph, or any downstream logic. **Stretch goal, time permitting:** wire `read_inbox` to a real, dedicated test Gmail account in read-only mode for the demo video, to show this swap-in working live rather than just describing it.
- **Concrete MCP contract, named so it doesn’t drift during implementation:**
 - `read_inbox(since: datetime | None) → list[RawItem]` — mock data lives in `mcp/fixtures/inbox.json`, a flat JSON array of objects matching the `RawItem` schema (§2.2.1); the mock server just reads and filters this file by `received_at`.
 - `read_hoa_portal(since: datetime | None) → list[RawItem]` — same shape, backed by

`mcp/fixtures/hoa_portal.json`, stub-served over a tiny local REST endpoint to simulate a real portal API rather than a flat file read, since that's closer to what a real HOA portal integration would look like.

- `read_invoices_folder()` → `list[RawItem]` — backed by a watched local folder (`mcp/fixtures/invoices/*.pdf`), each file's extracted text becoming one `RawItem` with `source="invoice"`.
- **Token passing:** the service token travels as a bearer token in an `Authorization` header on each MCP call (`Authorization: Bearer <token>`), matching how the real OAuth2 swap-in (below) would carry credentials — so the call-site code doesn't change shape when the mock token is replaced with a real one later.
- **Authentication/authorization model:** each MCP tool call carries a scoped, short-lived service token (mocked in v1 as a static dev token, designed to be swapped for an OAuth2 client-credentials flow in production). The MCP server validates the token and the requesting agent's identity before returning data. No MCP tool has write access in v1; all three are read-only, which is itself a security boundary (see §2.4).

Code execution environment:

- An ephemeral, sandboxed Python runtime tool that `TriageAgent` calls to mathematically audit vendor invoice totals (line-item sum vs. stated total) and cross-reference city notice dates against lease schedules. Sandboxed = no filesystem or network access from inside this execution context; it receives structured input, returns structured output, nothing else.

Agent-to-Agent (A2A) protocol — current scope:

The three agents above communicate through the ADK graph's internal message-passing, not A2A — A2A is for independent agents that don't share a single graph/process, and `IngestionAgent` / `TriageAgent` / `ResponseAgent` are sub-agents of one NightShift system. A2A becomes relevant if NightShift needs to talk to an agent *outside* its own graph — e.g., a vendor's own invoicing agent, or a separate `VendorNegotiationAgent` operated independently. That's documented as a clean v2 extension point (§2.6) rather than built now, since no second independent agent exists yet to justify it. This is a defensible, intentional scope line to state explicitly in the Writeup — judges should see you understood the distinction between internal multi-agent orchestration and A2A, rather than assuming they're interchangeable.

2.2.1 Data Flow Example — the hard case through the full pipeline

The graph diagram above shows structure; this shows it actually moving. Tracing the PRD \$1.5 hard-case fixture end to end is worth including verbatim in code comments or a docstring at the supervisor entrypoint — it's the fastest way for a judge skimming code to see the architecture is real, not just decorative boxes in a diagram:

```
1. IngestionAgent
   reads "the ceiling above my bathroom has a small water stain,
   has had it for a week, nothing dripping yet" via read_inbox (MCP)
   → emits RawItem(text=..., source="email", received_at=...)

2. TriageAgent
   - looks up tenant_email → property_id via the memory store (deterministic
     lookup, no LLM call — see §2.3)
   - calls gemini-1.5-flash with RawItem.text + property context
   - reasoning lands on RED (early structural water-damage signal, not
     just "no active leak = GREEN")
   - raw RawItem.text is pruned here — not carried forward (see §2.3)
   → emits ClassifiedItem(urgency="RED", property_id=..., summary=...)

3. ResponseAgent
   - receives ClassifiedItem only — no raw inbound text
   - calls gemini-1.5-pro to draft a tenant-facing acknowledgment +
     internal note recommending a plumber inspection
   - writes Draft(status="staged", ...) to the staging DB
   → nothing is sent; manager sees this in tomorrow's Morning Brief
```

Each arrow above is a real handoff object (`RawItem`, `ClassifiedItem`, `Draft`), not a conceptual gap — implementing these as explicit typed objects (Pydantic models are a natural fit) rather than passing loose dicts between agents is itself a small but visible signal of engineering care, and also gives the MCP swap-in (real Gmail/Outlook, §2.2) a stable contract to swap behind: `read_inbox` just needs to keep returning `RawItem` objects, regardless of what's behind it.

Concrete field definitions (named here so implementation doesn't drift or stall mid-build deciding this later):

```
class RawItem(BaseModel):
    id: str
    source: Literal["email", "hoa_portal", "invoice"]
    tenant_id: str | None          # None for HOA/vendor items not tied to a tenant
    raw_text: str
    received_at: datetime

class ClassifiedItem(BaseModel):
    id: str
    raw_item_id: str              # foreign key back to RawItem
    urgency_tier: Literal["RED", "YELLOW", "GREEN"]
    property_id: str
    summary: str                  # short generated summary, not the raw text
    classified_at: datetime

class Draft(BaseModel):
    id: str
    classified_item_id: str        # foreign key back to ClassifiedItem
    draft_text: str
    status: Literal["staged", "approved", "rejected", "snoozed", "ready_to_send"]
    approved_by: str | None       # non-null only once status == "approved"
    approved_at: datetime | None
```

`raw_text` deliberately lives only on `RawItem`, never copied onto `ClassifiedItem` or `Draft` — that's the field-level enforcement of the context-pruning behavior described above, not just a stated intention.

2.3 Context Engineering: Sessions, Skills & Memory (Day 3 Standards)

Sessions & state (Mandatory Feature #3 — Sessions & Context Engineering): Each overnight run initializes one ADK session, scoped to that night's batch and shared across all three agents in the graph so `TriageAgent` and `ResponseAgent` see the same run context `IngestionAgent` established. Session state (item count, per-agent progress, draft status) is held in a lightweight local layer (SQLite in v1) and is fully reconstructable if the process restarts mid-run — no overnight batch is silently lost.

Token budget / context window management strategy: This is the gap a naive build hits first — a portfolio with 50+ overnight items cannot be jammed into a single prompt, and a 3-agent pipeline makes this worse if handled carelessly (each agent re-reading full context from the last). NightShift's approach:

- Each item is classified **individually** by `TriageAgent` (one Gemini Flash call per item, small context) rather than batched into one giant prompt — this bounds per-call token cost regardless of inbox size.
- Property and lease context is **retrieved selectively** per item (only the relevant property's record is pulled from memory, not the entire portfolio) before `ResponseAgent` drafts — this is the actual "context engineering" the course unit is about, not just "use a big context window."
- **Explicit context compaction:** once `TriageAgent` finishes classifying an item, the raw inbound text is pruned from what gets passed forward — only the structured `ClassifiedItem` (urgency tier, matched property ID, a short generated summary) reaches `ResponseAgent`. The original message body is not needed for drafting and is not carried forward just because it's available; this is the concrete mechanism behind "agents pass structured handoff objects, not full conversation history" below.
- **Pruning vs. drafting quality (a real tension, resolved deliberately, not by accident):** pruning the raw message body keeps `ResponseAgent`'s context small, but drafting purely from a terse `ClassifiedItem.summary` risks a generic, robotic-sounding reply — which undermines the "professional, context-aware" drafting goal stated in the PRD §1. The resolution: `ResponseAgent`'s property-context lookup (from memory, not from the pruned raw text) includes a short property "personality" note — e.g., "older building, high-maintenance HVAC, tenant has been responsive in the past" — so drafts stay warm and specific without needing the original message body back. This is a small, deliberately-scoped piece of memory content, not a reason to abandon pruning.
- Agents pass only **structured handoff objects** (`RawItem` → `ClassifiedItem`) between graph nodes, not full conversation history — this is the context-compaction strategy: each agent gets exactly what it needs from the prior stage, not an ever-growing transcript.

- The Morning Brief itself is assembled programmatically (concatenating already-generated summaries), not regenerated by another LLM call over the full day's output — avoiding a second unnecessary large-context pass.

Modular agent skills — each ADK agent owns its own `Skill` folder with a `SKILL.md`, kept distinct from the agent-graph topology in §2.2 (agents are *who* does the work; skills are *how* — the internal capability modules each agent calls):

- `IngestionAgent/IngestionSkill/` — text/PDF scraping, initial document indexing, calls the three MCP read tools.
- `TriageAgent/ClassificationSkill/` — reasons through RED/YELLOW/GREEN thresholds, performs property matching.
- `ResponseAgent/ResponseDraftingSkill/` — generates context-aware, professional draft replies; never sends.

Long-term memory (Mandatory Feature #4 — Long-Term Memory): A lightweight Vector DB (or JSON store for v1 simplicity) retains historical property data, vendor terms, and tenant lease parameters across sessions, so context doesn't need to be re-derived from scratch each night. This memory store is shared infrastructure, read by `TriageAgent` (property matching) and `ResponseAgent` (drafting tone/history), but only ever written to by a separate nightly consolidation step — not directly by either agent mid-run — to avoid race conditions across the three concurrent-ish agent calls. **The matching mechanism is a stored schema lookup, not a model guess:** memory holds an explicit tenant-to-property mapping (tenant identifier → property ID → lease parameters), so `TriageAgent` resolves “which property does this item belong to” via a direct lookup against that mapping rather than asking Gemini to infer the property from address text in the message body each time — this is both cheaper (no LLM call needed for matching) and more reliable (a lookup either finds a record or doesn't; an LLM guess can be subtly wrong). Stated plainly, for the README and Writeup: **property matching uses a deterministic key lookup (tenant_email_or_id → property_id) rather than embedding full lease documents in every agent prompt — this eliminates hallucinated property assignments and keeps context windows small.** This is also a direct, concrete answer to a failure mode judges will likely have seen in other agent submissions (an LLM occasionally misattributing an item to the wrong property/customer because matching was left to the model instead of a lookup).

2.4 Agent Quality, Security & Guardrails (Day 4 Standards)

Framing — Effective Trust: Per the Day 4 course standard, security here isn't a one-time checklist but a continuously-maintained property of the system, since the agent's behavior is non-deterministic. NightShift's guardrails below are designed to be re-evaluated on every run, not verified once and assumed permanent.

- **Red/Blue/Green security triad** (one pillar within Effective Trust): Red = automated threat scans on incoming content, blocking prompt-injection vectors disguised as malicious tenant emails or corrupted PDFs before they reach the classification prompt. Blue/Green = output validation layers protecting against data leakage (e.g., one tenant's lease data appearing in another tenant's draft) and formatting failures.
- **PII handling:** tenant names, unit addresses, and lease terms are personally identifiable information. NightShift scopes every memory retrieval to the single property/tenant being processed (no cross-tenant context bleed), and the redaction list used in observability explicitly includes tenant PII fields, not just API keys.
- **Slopsquatting & dependency shields:** mandatory scanning of all external Python packages before they're added to the project, to catch hallucinated/typosquatted package names — a known vibe-coding-specific risk when dependencies are added via natural-language prompts.
- **Evaluation loops:** a regression test suite — `tests/eval_urgency.py`, run via `agents-cli test` or plain `pytest` — checks classification accuracy against 20+ synthetic inbox fixtures on every change, including the PRD \$1.5 hard case by name, so a code change that breaks the borderline-case judgment (not just the easy cases) is caught before it ships, not just before a demo. **What the fixture set actually contains** (concrete, not just “20 items”): roughly 6–8 unambiguous RED items (active leak, no-heat report, city violation notice), 6–8 unambiguous GREEN items (lightbulb out, routine HOA newsletter), 4–6 unambiguous YELLOW items (invoice total mismatch, inspection due in 5 days), and the one named hard case from PRD \$1.5 with its ground-truth label set to RED — each fixture is a `(raw_text, expected_urgency_tier)` pair, stored as the same JSON shape as the MCP mock fixtures (§2.2) so the same data format is reused rather than inventing a second one just for tests. **Make the fixtures realistically messy, not clean:** a few items should include typos, run-on sentences, or a slightly garbled HOA-notice format (as a real scanned/forwarded notice often is) rather than tidy, well-formatted prose — this is cheap to do (an hour of writing fixtures) and makes `TriageAgent`'s reasoning look meaningfully more impressive in the demo video than a model classifying clean, textbook-perfect inputs would. **Evaluation metric, made concrete:** since RED/YELLOW/GREEN is a 3-class classification problem, the test harness should report a full confusion matrix against the labeled fixture set,

not just a single accuracy percentage — accuracy alone can hide a model that’s good at GREEN/RED but consistently confuses YELLOW with GREEN, which a confusion matrix surfaces immediately and a single number doesn’t. The false-RED and false-GREEN rates already in the PRD’s §1.3 Success Metrics are themselves two cells read directly off this matrix, so this isn’t new measurement infrastructure — it’s reporting what’s already being computed, in the form that actually reveals where errors cluster.

Threat model — what happens when something goes wrong, not just what prevents it: the mechanisms above describe prevention; a threat model states the failure scenario and the actual response, which is what “Effective Trust” means in practice rather than as a label:

- **If the MCP server is compromised or returns malicious data:** because all three MCP tools are read-only (§2.2), a compromised MCP server can feed bad *input* to `IngestionAgent` but cannot itself send anything or write anything — the worst case is a bad classification, not an unauthorized action, because the HITL state machine (§2.5) sits between any agent output and any real-world effect regardless of how that output was produced.
- **If a prompt-injection attempt slips past the Red-team content scan:** the Blue/Green output validation layer is the second line of defense — even if injected text influences `TriageAgent`’s reasoning, the worst plausible outcome is a wrong urgency tier or a malformed draft, not an unauthorized send, since drafting wrong is still bounded by the same staged-until-approved guarantee. This is the concrete payoff of the headline hook (PRD §0.5.1): the safety property doesn’t depend on the injection scanner catching everything, because there’s a second, structural backstop behind it.
- **If `ResponseAgent` is manipulated into drafting something inappropriate (e.g., an injected instruction asking it to draft a different message than intended):** the manager sees the actual draft text before approving — HITL isn’t just a gate on *sending*, it’s a human reading the real content, so a manipulated draft is caught at review rather than ever reaching a tenant.
- **What this threat model does *not* cover (named explicitly, not silently skipped):** a compromised manager credential approving a malicious draft is out of scope for v1 — that’s an identity/access-management problem, not an agent-architecture problem, and is appropriately deferred rather than pretended to be solved here.

2.5 Prototype to Production: Spec-Driven Development (Day 5 Standards)

Day 5 of the course is specifically about **Spec-Driven Development (SDD)**, not just observability and deployment plumbing.

Spec-Driven Development: Behavior is defined first as Gherkin-style feature specs, treated as the source of truth; the generated code is disposable and regenerable against the spec. Example:

```
# features/urgency_classification.feature
Feature: Overnight item urgency classification

  Scenario: Structural damage report is classified RED
    Given an overnight item mentioning "ceiling water leak"
    When ClassificationSkill processes the item
    Then the urgency tier should be "RED"
    And the item should be flagged for immediate manager attention

  Scenario: Routine maintenance request is classified GREEN
    Given an overnight item mentioning "lightbulb replacement needed"
    When ClassificationSkill processes the item
    Then the urgency tier should be "GREEN"

  Scenario: Drafted response never auto-sends
    Given any classified item with a generated draft response
    When the draft is created
    Then the draft status should be "staged"
    And no outbound message should be transmitted
    And the status should only change to "approved" via explicit manager action
```

- **Code-review agent / Policy Server:** an automated review agent checks each generated code change against the Gherkin specs before it’s merged; a Policy Server enforces non-negotiable constraints (e.g., “no code path may set status directly to `sent`”) independently of the main agent’s own reasoning — this is the “hybrid” check the course

describes: deterministic policy enforcement sitting alongside probabilistic agent behavior.

Observability: OpenTelemetry tracing records the agent’s step-by-step reasoning trajectory — the exact graph path taken to mark a given notice RED — so any classification can be audited after the fact. Concretely, each span captures:

- **Model call path:** which model handled the call (`gemini-1.5-flash` for classification, `gemini-1.5-pro` for drafting/ambiguous cases), so a slow or expensive run can be traced back to which agent made which model choice.
- **Token usage per call:** input/output token counts per agent invocation, rolled up to a per-item and per-overnight-run total — this is what actually lets the “token cost per overnight run” success metric (PRD §1.3) be measured rather than estimated.
- **Tool execution latency:** timing for each MCP tool call (`read_inbox` , `read_hoa_portal` , `read_invoices_folder`) and each sandboxed code-execution call (invoice audit, lease date cross-reference), so a slow external mock endpoint or a slow audit calculation is visible in the trace rather than hidden inside an opaque “ingestion took 4 seconds” span.
- As stated above, secrets are never logged in any of these attributes — the redaction list applies to trace data the same as it does to application logs.

Human triage state machine: the staging database schema enforces valid transitions only:

```
draft_created → staged → approved → ready_to_send
                ↓
                rejected
                ↓
                snoozed → staged (re-enters queue)
```

No row can move from `staged` directly to `ready_to_send`; the `approved` state requires a non-null `approved_by` (manager identity) and `approved_at` timestamp, checked at the database constraint level — not just in application code — so this guarantee holds even if the agent itself is compromised or misbehaves. **Concrete implementation:** this is a SQLAlchemy model with a `CHECK` constraint on the status column restricting it to the five valid states, plus a trigger or application-layer validator (a small finite-state-machine validator function is sufficient — no need for a full FSM library at this scale) that rejects any transition not in the table above before the write commits. The point being made to a judge here is that “enforced at the database layer” isn’t just a phrase in this document — it names the actual mechanism, which is buildable in an afternoon and worth building first (see the Build Priority Order below).

2.6 Deployment Evidence (optional — counts toward Technical Implementation, not a separate category)

Note: there is no standalone “Deployment Demonstration” rubric category. Kaggle’s official rubric states plainly that participants are not required to deploy their agents to a live public endpoint for judging purposes; however, if you do deploy, please provide documentation to reproduce the deployment. Deployment readiness is one input into the 50-point Technical Implementation score, not a points bucket on its own — so this section is worthwhile-but-optional polish, not a graded requirement to chase.

If you do include it (recommended — it’s cheap relative to the rest of the build and strengthens the “architecture quality” read of Technical Implementation):

- A `Dockerfile` in the repo root that builds the full 3-agent graph into a single container, runnable via `docker run` with env vars for the Gemini API key and MCP endpoint URLs.
- A `cloudbuild.yaml` or equivalent deploy script targeting Google Cloud Run — including it costs little even if it’s never been triggered against a live billing account.
- If you do deploy live, documentation sufficient to reproduce the deployment is explicitly required by the rubric — don’t deploy and then omit the steps.
- A short “Scaling Notes” subsection in the README (PRD §3.4) stating how multiple properties/portfolios would be handled and what would need to change to support concurrent overnight runs (the SQLite state layer is the v1-only bottleneck named here, with a managed Postgres/Firestore swap-in as the production path).

2.7 Architecture Decision Log

Decision	Reasoning
Multi-agent supervisor graph (<code>IngestionAgent</code> / <code>TriageAgent</code> / <code>ResponseAgent</code>), not a single agent with internal skills	The official Key Concepts checklist lists “Agent / Multi-agent system (ADK)” as a graded item demonstrated in code, with this exact split as the natural worked example. The task itself (ingest → classify → draft) would also work as one agent with three internal skills, but separate agents make the graph topology visibly inspectable by a judge reviewing code
A2A protocol not used between the three internal agents	A2A is for independent agents outside a shared graph/process; these three share one ADK graph and session, so internal message-passing is the correct mechanism, not A2A. A2A is reserved for a genuinely external agent (e.g. a vendor’s own agent) — see §2.2
Read-only MCP tools only	Removes an entire class of risk (no write/send capability exists at the tool layer, independent of any prompt-level safeguard)
SQLite for v1 state, Vector DB path open for memory	Keeps v1 simple to run in Cursor locally; schema designed to be portable to a managed DB at production hardening time — named as the scaling bottleneck in §2.6
Deployment artifacts (Dockerfile, deploy script) included despite not being required	Costs little, strengthens the Technical Implementation read of the architecture, and is the right call given the rubric explicitly allows but doesn’t mandate it

2.8 Build Priority Order (for the 8-day window)

Build in this order, not file-by-file or agent-by-agent in isolation — each step should produce something runnable before moving to the next, so there’s always a working (if incomplete) system rather than three half-finished agents on the deadline:

1. **Supervisor graph + three stub agents** — wire the ADK graph topology first with agents that pass data through untouched (no real LLM calls yet). This proves the plumbing before any intelligence is added.
2. **MCP mock tools + `IngestionAgent`** — get `read_inbox` / `read_hoa_portal` / `read_invoices_folder` returning real mock data, `IngestionAgent` producing real `RawItem` objects.
3. **`TriageAgent` + classification skill + memory lookup** — including the PRD §1.5 hard case as a named test from day one, not added later.
4. **Human triage state machine + staging DB** — build the §2.5 SQLAlchemy/CHECK-constraint enforcement *before* `ResponseAgent` exists to write to it, so the safety guarantee is never temporarily absent at any point in the build.
5. **`ResponseAgent` + Morning Brief assembly** — drafting and the manager-facing view.
6. **Observability + eval harness** — `tests/eval_urgency.py` and the OpenTelemetry spans from §2.5.
7. **Polish** — README, video assets, optional Dockerfile.

The state machine is deliberately sequenced *before* the agent that would write to it (step 4 before step 5 is complete) rather than after, since it’s the project’s core differentiator — building it early means it’s never the thing left unfinished if time runs short.

De-risking note on sandboxed code execution (§2.2): a fully isolated sandbox runtime is non-trivial to get exactly right in a short build window. If it threatens to block progress on the core agent logic above, the acceptable fallback for v1 is a plain restricted Python function/module (no actual process isolation) clearly labeled as a placeholder sandbox boundary in code comments — the invoice-audit *logic* (line-item sum vs. stated total) is what’s actually being demonstrated for the “code execution environment” checklist item, and a true sandbox can be named as a v2 hardening step rather than something that’s allowed to stall the agents that matter more to the rubric.

End of TDD. See the companion Product Requirements Document (PRD) for value proposition, scope, success metrics, competitive positioning, and submission deliverables (Writeup, video, README, legal constraints).